

Rapport de laboratoire 3

MTH8408

Yasmine Amami

```
using Pkg
Pkg.activate("labo3_env")
Pkg.add("ADNLPModels")
Pkg.add("NLPModels")
Pkg.add("LDLFactorizations")
Pkg.add("Krylov")
using LinearAlgebra, Printf, ADNLPModels, NLPModels, LDLFactorizations, Krylov
```

Contexte

Dans ce laboratoire, on demande d'implémenter deux méthodes basées sur la méthode de Newton pour le problème

$$\min_x f(x) \quad (1)$$

où $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est deux fois continûment différentiable.

Question 1

En cours, nous avons vu la méthode de Newton modifiée avec recherche linéaire inexacte pour résoudre (1).

Dans cette question, on demande d'implémenter et de tester cette méthode *en utilisant une factorisation modifiée* de $\nabla^2 f(x_k)$.

Votre implémentation doit avoir les caractéristiques suivantes :

1. prendre un `AbstractNLPModel` en argument ;
2. un critère d'arrêt absolu et relatif sur le gradient de l'objectif ;
3. un critère d'arrêt portant sur le nombre d'itérations (le nombre maximum d'itérations devrait dépendre du nombre de variables n du problème) ;
4. toujours démarrer de l'approximation initiale spécifiée par le modèle ;
5. allouer un minimum en utilisant les opérations vectorisées (`.*`, `.+`, `.*=`, etc.) autant que possible ;
6. votre fonction principale doit être documentée—reportez-vous à <https://docs.julialang.org/en/v1/manual/documentation> ;
7. votre fonction doit faire afficher les informations pertinentes à chaque itération sous forme de tableau comme vu en cours.

Tester votre implémentation sur le problème polynomial vu en classe et les problèmes non linéaires de la section *Problèmes test* ci-dessous.

```
"""
```

```
    newton_modifiee(model, eps_a=1.0e-5, eps_r=1.0e-5)
```

```
Résout un problème d'optimisation non linéaire sans contraintes en utilisant la méthode de Newton modif.
```

```

# Arguments
- `model`: Un modèle d'optimisation non linéaire conforme à l'interface AbstractNLPModel
- `eps_a`: Critère d'arrêt absolu sur la norme du gradient (par défaut: 1.0e-5)
- `eps_r`: Critère d'arrêt relatif sur la norme du gradient (par défaut: 1.0e-5)
"""
function newton_modifiee(model, eps_a=1.0e-5, eps_r=1.0e-5)
    x = model.meta.x0
    n = length(x)
    g0 = grad(model, x)
    seuil = eps_a + eps_r * norm(g0)

    println(" iter   | f(x)           | |grad| | pas | slope")
    println("-----|-----|-----|-----|-----")

    for iter in 1:10*n
        f = obj(model, x)
        g = grad(model, x)
        if norm(g) <= seuil
            break
        end

        H = Symmetric(triu(hess(model, x)))
        # On utilise la factorisation LDL modifiée
        LDL = ldl_analyze(H)
        LDL.tol = Inf
        LDL.r1 = 1.0e-5
        LDL = ldl_factorize!(H, LDL)
        d = -LDL \ g

        # On utilise Armijo
        t = 1.0
        slope = dot(d, g)
        while obj(model, x + t*d) > f + 1e-4 * t * slope
            t *= 0.5
        end

        x += t*d

        @printf("%4d   | %8.2e | %10.2e | %6.2e | %6.2e\n", iter, f, norm(g), t, slope)
    end
    return x
end

```

Main.Notebook.newton_modifiee

Question 2

Dans cette question, on demande d'implémenter la méthode de Newton inexacte pour résoudre (1).

Votre implémentation doit avoir les mêmes caractéristiques qu'à la question 1.

Il faut de plus ajuster votre méthode de manière à encourager la convergence locale superlinéaire.

Tester votre implémentation sur le problème polynomial vu en classe et les problèmes non linéaires de la

section *Problèmes test* ci-dessous.

```
"""
    newton_inexacte(model, eps_a=1.0e-5, eps_r=1.0e-5; verbose=true)

Résout un problème d'optimisation non linéaire sans contraintes en utilisant
la méthode de Newton inexacte avec un solveur itératif pour les systèmes linéaires.

# Arguments
- `model`: Un modèle d'optimisation non linéaire conforme à l'interface AbstractNLPModel
- `eps_a`: Critère d'arrêt absolu sur la norme du gradient (par défaut: 1.0e-5)
- `eps_r`: Critère d'arrêt relatif sur la norme du gradient (par défaut: 1.0e-5)
"""
function newton_inexacte(model, eps_a=1.0e-5, eps_r=1.0e-5)
    x = model.meta.x0
    n = length(x)
    g0 = grad(model, x)
    seuil = eps_a + eps_r * norm(g0)

    println(" iter | f(x) | |grad| | pas | slope | ")
    println("-----|-----|-----|-----|-----|")

    for iter in 1:10*n
        f = obj(model, x)
        g = grad(model, x)
        if norm(g) <= seuil
            break
        end

        H = hess(model, x)
        # On définit pour encourager la convergence superlinéaire
        = min(0.5, sqrt(norm(g)))
        # On résout le système avec CG
        d, stats = Krylov.cg(H, -g, rtol=, itmax=n, linesearch=true)
        # On utilise ici Armijo
        t = 1.0
        slope = dot(d, g)
        while obj(model, x + t*d) > f + 1e-4 * t * slope
            t *= 0.5
        end

        x += t * d

        @printf("%4d | %8.2e | %10.2e | %6.2e | %6.2e | %6.2e\n", iter, f, norm(g), t, slope, )
    end
    return x
end
```

Main.Notebook.newton_inexacte

Résultats numériques

Problèmes test

Votre premier problème test sera le problème polynomial vu en classe.

```
f(x) = 2*x[1]^3 - 3*x[1]^2 - 6*x[1]*x[2]* (x[1] - x[2] - 1)
model = ADNLPMModel(f, [1.5, 0.5])
result1 = newton_modifiee(model)
result2 = newton_inexacte(model)
```

iter	f(x)	grad	pas	slope
1	0.00e+00	4.50e+00	1.00e+00	-1.69e+00
2	-9.49e-01	8.44e-01	1.00e+00	-9.49e-02
3	-1.00e+00	7.59e-02	1.00e+00	-9.38e-04
4	-1.00e+00	9.15e-04	1.00e+00	-1.40e-07

iter	f(x)	grad	pas	slope
1	0.00e+00	4.50e+00	1.00e+00	-1.12e+00
2	-5.62e-01	1.88e+00	1.00e+00	-7.81e-01
3	-9.89e-01	2.60e-01	1.00e+00	-2.09e-02
4	-1.00e+00	9.63e-03	1.00e+00	-3.08e-05

```
2-element Vector{Float64}:
 1.0000051200131073
 2.560006553644755e-6
```

Utiliser ensuite les problèmes non linéaires du dépôt `OptimizationProblems.jl` qui sont sans contraintes et ont 100 variables. Vous pouvez y accéder à l'aide de l'extrait de code suivant :

```
Pkg.add("OptimizationProblems") # collection + outils pour sélectionner les problèmes
using OptimizationProblems, OptimizationProblems.ADNLPProblems

meta = OptimizationProblems.meta
problem_list = meta[(meta.ncon.==0) .& .!meta.has_bounds .& (meta.nvar.==100), :name]
problems = (OptimizationProblems.ADNLPProblems.eval(Meta.parse(problem))() for problem in problem_list)
```

Parmi ces problèmes, choisissez-en 3 et illustrez le comportement de chacune des méthodes.

Validation de la méthode de Newton modifiée

```
test3 = OptimizationProblems.ADNLPProblems.brybnd()
x3 = newton_modifiee(test3)
```

iter	f(x)	grad	pas	slope
1	1.80e+03	1.37e+03	1.00e+00	-2.43e+03
2	3.32e+02	4.10e+02	1.00e+00	-4.84e+02
3	4.45e+01	1.11e+02	1.00e+00	-7.26e+01
4	3.01e+00	2.34e+01	1.00e+00	-5.50e+00
5	5.83e-02	2.56e+00	1.00e+00	-1.11e-01
6	4.90e-04	1.60e-01	1.00e+00	-9.66e-04

```
100-element Vector{Float64}:
-0.4284306846051624
-0.47664830675709824
-0.519671031032967
-0.5581101515162381
-0.5925145728473755
-0.624511136234421
```

-0.6232398597111274
-0.6214193587715467
-0.6196153522211775
-0.6182256210322281

-0.6180339938469783
-0.6180339939190225
-0.6180339939886552
-0.618033994875833
-0.6180339682622509
-0.618034778063966
-0.618008241921486
-0.6188732817558193
-0.5862791272704341

Validation de la méthode de Newton tronquée

```
test1 = OptimizationProblems.ADNLPProblems.srosenbr()
x1 = newton_inexacte(test1)
test2 = OptimizationProblems.ADNLPProblems.argtrig()
x2 = newton_inexacte(test2)
test3 = OptimizationProblems.ADNLPProblems.brybnd()
x3 = newton_inexacte(test3)
```

iter	f(x)	grad	pas	slope	
-----	-----	-----	-----	-----	---
1	1.21e+03	1.65e+03	1.00e+00	-1.80e+03	5.00e-01
2	2.28e+02	2.19e+02	1.00e+00	-4.31e+01	5.00e-01
3	2.06e+02	1.38e+01	1.00e+00	-1.05e+00	5.00e-01
4	2.06e+02	2.98e+01	1.00e+00	-1.05e+00	5.00e-01
5	2.05e+02	1.39e+01	1.00e+00	-1.01e+00	5.00e-01
6	2.05e+02	2.90e+01	1.00e+00	-1.01e+00	5.00e-01
7	2.04e+02	1.40e+01	1.00e+00	-9.82e-01	5.00e-01
8	2.04e+02	2.83e+01	1.00e+00	-9.81e-01	5.00e-01
9	2.03e+02	1.41e+01	1.00e+00	-9.55e-01	5.00e-01
10	2.03e+02	2.76e+01	1.00e+00	-9.55e-01	5.00e-01
11	2.02e+02	1.42e+01	1.00e+00	-9.32e-01	5.00e-01
12	2.02e+02	2.70e+01	1.00e+00	-9.32e-01	5.00e-01
13	2.01e+02	1.43e+01	1.00e+00	-9.11e-01	5.00e-01
14	2.01e+02	2.65e+01	1.00e+00	-9.11e-01	5.00e-01
15	2.01e+02	1.44e+01	1.00e+00	-8.92e-01	5.00e-01
16	2.00e+02	2.59e+01	1.00e+00	-8.93e-01	5.00e-01
17	2.00e+02	1.45e+01	1.00e+00	-8.75e-01	5.00e-01
18	1.99e+02	2.55e+01	1.00e+00	-8.77e-01	5.00e-01
19	1.99e+02	1.47e+01	1.00e+00	-8.61e-01	5.00e-01
20	1.98e+02	2.50e+01	1.00e+00	-8.63e-01	5.00e-01
21	1.98e+02	1.48e+01	1.00e+00	-8.48e-01	5.00e-01
22	1.97e+02	2.46e+01	1.00e+00	-8.50e-01	5.00e-01
23	1.97e+02	1.49e+01	1.00e+00	-8.36e-01	5.00e-01
24	1.97e+02	2.42e+01	1.00e+00	-8.39e-01	5.00e-01
25	1.96e+02	1.50e+01	1.00e+00	-8.26e-01	5.00e-01
26	1.96e+02	2.38e+01	1.00e+00	-8.29e-01	5.00e-01
27	1.95e+02	1.51e+01	1.00e+00	-8.17e-01	5.00e-01
28	1.95e+02	2.34e+01	1.00e+00	-8.20e-01	5.00e-01

29	1.95e+02	1.53e+01	1.00e+00	-8.09e-01	5.00e-01
30	1.94e+02	2.30e+01	1.00e+00	-8.12e-01	5.00e-01
31	1.94e+02	1.54e+01	1.00e+00	-8.02e-01	5.00e-01
32	1.93e+02	2.27e+01	1.00e+00	-8.06e-01	5.00e-01
33	1.93e+02	1.55e+01	1.00e+00	-7.96e-01	5.00e-01
34	1.93e+02	2.24e+01	1.00e+00	-8.00e-01	5.00e-01
35	1.92e+02	1.57e+01	1.00e+00	-7.91e-01	5.00e-01
36	1.92e+02	2.21e+01	1.00e+00	-7.96e-01	5.00e-01
37	1.91e+02	1.58e+01	1.00e+00	-7.87e-01	5.00e-01
38	1.91e+02	2.18e+01	1.00e+00	-7.92e-01	5.00e-01
39	1.91e+02	1.59e+01	1.95e-03	-2.96e+04	5.00e-01
40	1.71e+02	2.12e+02	1.00e+00	-7.93e+01	5.00e-01
41	1.30e+02	1.88e+01	6.25e-02	-5.23e+02	5.00e-01
42	1.07e+02	8.49e+01	1.00e+00	-2.17e+01	5.00e-01
43	9.56e+01	1.56e+01	6.25e-02	-7.33e+02	5.00e-01
44	9.19e+01	1.26e+02	1.00e+00	-7.89e+01	5.00e-01
45	5.21e+01	1.45e+01	1.25e-01	-1.34e+02	5.00e-01
46	4.01e+01	3.66e+01	1.00e+00	-6.90e+00	5.00e-01
47	3.66e+01	1.23e+01	2.50e-01	-7.96e+01	5.00e-01
48	3.39e+01	9.01e+01	1.00e+00	-2.55e+01	5.00e-01
49	2.09e+01	7.72e+00	5.00e-01	-2.27e+01	5.00e-01
50	1.64e+01	6.26e+01	1.00e+00	-9.12e+00	5.00e-01
51	1.18e+01	4.85e+00	5.00e-01	-1.50e+01	5.00e-01
52	8.62e+00	5.57e+01	1.00e+00	-5.50e+00	5.00e-01
53	5.84e+00	3.00e+00	5.00e-01	-8.43e+00	5.00e-01
54	3.68e+00	3.96e+01	1.00e+00	-2.27e+00	5.00e-01
55	2.54e+00	1.75e+00	5.00e-01	-4.14e+00	5.00e-01
56	1.29e+00	2.38e+01	1.00e+00	-7.04e-01	5.00e-01
57	9.37e-01	9.72e-01	5.00e-01	-1.67e+00	5.00e-01
58	3.69e-01	1.10e+01	1.00e+00	-1.37e-01	5.00e-01
59	3.00e-01	5.22e-01	1.00e+00	-5.64e-01	5.00e-01
60	1.41e-01	1.66e+01	1.00e+00	-2.77e-01	5.00e-01
61	2.32e-03	6.01e-02	1.00e+00	-4.61e-03	2.45e-01
62	1.05e-05	1.43e-01	1.00e+00	-2.05e-05	3.78e-01
iter	f(x)	grad	pas	slope	
-----	-----	-----	-----	-----	---
1	7.52e-01	5.82e+00	1.00e+00	-3.38e+01	5.00e-01
2	-4.33e+02	1.78e+02	1.00e+00	-3.16e+04	5.00e-01
3	-4.50e+03	4.01e+02	2.50e-01	-1.61e+05	5.00e-01
4	-5.37e+03	3.79e+02	1.25e-01	-1.33e+05	5.00e-01
5	-5.66e+03	4.15e+02	2.50e-01	-1.88e+04	5.00e-01
6	-7.82e+03	3.49e+02	1.00e+00	-3.22e+03	5.00e-01
7	-8.55e+03	3.37e+02	1.00e+00	-2.73e+03	5.00e-01
8	-9.19e+03	2.84e+02	1.00e+00	-1.37e+03	5.00e-01
9	-9.70e+03	1.28e+02	1.00e+00	-2.27e+02	5.00e-01
10	-9.81e+03	5.46e+01	1.00e+00	-1.36e+02	5.00e-01
11	-9.85e+03	5.22e+01	5.00e-01	-2.36e+02	5.00e-01
12	-9.89e+03	2.91e+01	1.00e+00	-2.33e+01	5.00e-01
13	-9.90e+03	1.41e+01	1.00e+00	-2.60e+00	5.00e-01
14	-9.90e+03	5.66e+00	1.00e+00	-2.89e+00	5.00e-01
15	-9.90e+03	1.22e+00	1.00e+00	-3.56e-02	5.00e-01
16	-9.90e+03	4.04e-01	1.00e+00	-1.67e-02	5.00e-01
17	-9.90e+03	1.54e-01	1.00e+00	-1.77e-03	3.93e-01
18	-9.90e+03	5.94e-02	1.00e+00	-1.63e-04	2.44e-01

19		-9.90e+03		1.11e-02		1.00e+00		-1.77e-05		1.05e-01
20		-9.90e+03		9.23e-04		1.00e+00		-7.37e-08		3.04e-02
iter		f(x)		grad		pas		slope		
-----		-----		-----		-----		-----		---
1		1.80e+03		1.37e+03		1.00e+00		-2.43e+03		5.00e-01
2		3.34e+02		4.10e+02		1.00e+00		-4.86e+02		5.00e-01
3		4.53e+01		1.12e+02		1.00e+00		-7.33e+01		5.00e-01
4		3.32e+00		2.39e+01		1.00e+00		-5.81e+00		5.00e-01
5		1.97e-01		4.05e+00		1.00e+00		-3.26e-01		5.00e-01
6		1.77e-02		1.13e+00		1.00e+00		-3.32e-02		5.00e-01
7		2.48e-04		1.19e-01		1.00e+00		-4.86e-04		3.45e-01
8		2.21e-06		1.57e-02		1.00e+00		-4.31e-06		1.25e-01

100-element Vector{Float64}:

```
-0.42835615775366254
-0.4766061059074351
-0.5196594727040306
-0.5581052459233601
-0.5925227206419715
-0.6245046816107084
-0.6232364813672939
-0.6214182235035315
-0.6196140119474483
-0.6182231816285018
```

```
-0.6180340524151045
-0.6180322156643081
-0.6180289778552042
-0.6180304565354233
-0.6180313524369173
-0.6180338572085463
-0.6180086313515497
-0.6188789772282592
-0.586288653217329
```

Commentaires sur les résultats

- Au niveau de la vitesse de convergence, on constate que la méthode de Newton modifiée semble converger en moins d'itérations sur les problèmes de petite taille, comme le problème polynomial. Toutefois, il faut noter que la méthode modifiée requiert plus de ressources mémoire pour la factorisation complète du Hessien.
- La méthode de Newton inexacte peut parfois nécessiter plus d'itérations internes du solveur CG pour des problèmes mal conditionnés, mais l'adaptation de `CG` permet d'améliorer la convergence.
- Les deux méthodes montrent une réduction rapide de la norme du gradient près de la solution optimale.